VOLUME XX, 2024
Received xxxx 00, 0000, accepted xxxx 00, 0000, date of publication xxxx 00, 0000.
Digital Object Identifier 10.1109/ACCESS.2024.XXXXXXX

Smart Delivery Route Optimizer: A Multi-Algorithm Comparative Study Applying 0/1 Knapsack, Dijkstra Shortest Path, and Genetic Algorithms to Indian Urban Food Delivery Networks

Tanisha Ruhi[1], **Rishav Kumar Pankaj**[2], **and Ahana Ray**[3]
[123]School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore 632014, India
Corresponding author: Rishav Kumar Pankaj (e-mail: rishavkumar.pankaj2024@vitstudent.ac.in)
This work was completed as part of the Design and Analysis of Algorithms (DAA) course project at VIT Vellore, 2024.

ABSTRACT This paper presents a comprehensive multi-algorithm framework for profit-maximising order selection and route optimisation in urban food delivery networks, with specific application to Bangalore, India. The core optimisation problem is formalised as a 0/1 Knapsack Integer Linear Programme (ILP) under time-budget constraints, incorporating zone-specific traffic multipliers, Service-Level Agreement (SLA) penalty functions, and stochastic cancellation probabilities. Six algorithms are implemented, benchmarked, and rigorously compared: (i) Greedy O(n log n), (ii) Dynamic Programming O(n×T) with a corrected strict-comparison traceback eliminating duplicate selection on tie values, (iii) a Greedy-DP Hybrid achieving ≥98% of optimal profit at 40% reduced state-space, (iv) a Genetic Algorithm with proper tournament selection, elitism, and a feasibility repair operator, (v) First-Come-First-Served (FCFS) as a real-world manual baseline, and (vi) Nearest-First distance baseline. A space-optimised 1D DP reducing memory from O(n×T) to O(T) is formally derived and implemented. After order selection, a Nearest-Neighbour Travelling Salesman Problem (TSP) heuristic sequences deliveries over Dijkstra shortest-paths on a zonal road graph, completing the end-to-end pipeline. Empirical benchmarks across n = 4 to 20 orders with T = 50 minutes demonstrate that DP achieves a guaranteed global optimum while the Hybrid achieves ≥98–100% of DP profit. Sensitivity analysis reveals that the greedy gap widens with increasing n and traffic severity. A practical algorithm selector decision tree grounded in empirical breakeven thresholds is provided. Results indicate that DP-based dispatch increases per-trip earnings by ₹10–45 compared to FCFS manual dispatch, equivalent to ₹500–2,250 daily improvement per rider at 50 trips/day.
INDEX TERMS *0/1 knapsack problem, algorithm design and analysis, Dijkstra shortest path, dynamic programming, food delivery optimisation, genetic algorithm, greedy algorithm, integer linear programming, last-mile logistics, multi-objective optimisation, NP-hardness, SLA penalty model, travelling salesman problem, urban traffic modelling.*

This work is licensed under a Creative Commons Attribution 4.0 License.

I. INTRODUCTION

Urban food delivery platforms such as Zomato and Swiggy collectively process over 7.5 million orders daily in India [1]. Each delivery rider must select a subset of available orders that maximises net earnings within a fixed time window, accounting for zone-specific traffic conditions, SLA deadlines, and stochastic cancellations. This decision problem is formally equivalent to the 0/1 Knapsack Problem [2], which is NP-Hard in the general case.

Existing systems rely on FCFS manual dispatch or simple greedy heuristics. Neither provides optimality guarantees, nor accounts for the full complexity of real-world delivery networks including heterogeneous zone traffic, SLA penalty structures, and efficient route sequencing. This paper addresses all these gaps within a unified algorithmic framework.

Specific contributions: (1) formal ILP problem formulation with capped SLA penalty model; (2) corrected DP traceback using strict comparison; (3) O(T) space-optimised 1D DP; (4) corrected GA with tournament selection, elitism, and feasibility repair operator; (5) end-to-end integration of Knapsack → Dijkstra → TSP; (6) empirical benchmarks with measured runtimes; (7) sensitivity analysis across traffic and SLA configurations; and (8) algorithm selector decision tree.

II. RELATED WORK

The 0/1 Knapsack Problem has been studied since Bellman's dynamic programming formulation [3]. Dantzig established the fractional relaxation [4], while Karp proved NP-Hardness of the integer variant [5]. Practical DP solutions with O(n×T) complexity are covered in Cormen et al. [6] and Kleinberg and Tardos [7].

In logistics, Vehicle Routing Problems (VRP) represent the spatial extension of assignment problems [8]. Dijkstra's algorithm [9] provides optimal shortest-path computation. The Travelling Salesman Problem governing delivery sequencing is also NP-Hard [10]; nearest-neighbour heuristics provide a 2-approximation [11]. Genetic Algorithms for combinatorial optimisation were formalised by Holland [12], with empirical hyperparameter guidelines studied in [13].

III. PROBLEM FORMULATION

A. Single-Rider 0/1 Knapsack ILP

Let n be the number of available orders. For order i, define: p_i (rider profit, INR), t_i (base delivery time, min), z_i (zone), λ_i (zone traffic multiplier), T (time budget), SLA_i (deadline), $x_i \in \{0,1\}$ (selection variable). Adjusted time: $\tilde{t}_i = t_i \times \lambda_i$. Net profit: $net_i = p_i - penalty(\tilde{t}_i, SLA_i)$.

$$\text{maximise } \Sigma_i\, net_i \cdot x_i$$
$$\text{s.t. } \Sigma_i\, \tilde{t}_i \cdot x_i \le T, \quad x_i \in \{0,1\} \quad (1)$$

B. SLA Penalty Model (Capped)

The piecewise SLA penalty with cap prevents unrealistic divergence:

$$\text{penalty} = \begin{cases} 0 & \text{if } \tilde{t} \le SLA \\ \alpha(\tilde{t} - SLA) & \text{if } SLA < \tilde{t} \le 1.5 \cdot SLA \\ \min(\beta e^{\tilde{t}/SLA}, 150) & \text{if } \tilde{t} > 1.5 \cdot SLA \end{cases} \quad (2)$$

where α = ₹1.5/min, β = ₹8, CAP = ₹150. SLA: Express=20, High=30, Medium=45, Low=60 min.

C. Zone Traffic Model

$$\lambda(zone) = 1.0 + 0.40 \cdot [rain \cap exposed] + 0.30 \cdot [peak \cap isPeak] + 0.25 \cdot [jam \cap signals] + 0.20 \cdot [market \cap hasMarket] + 0.15 \cdot [narrow \cap isNarrow] \quad (3)$$

IV. ALGORITHMS

A. Greedy Algorithm

Sort orders by $net_i/\tilde{t}_i$ ratio descending; greedily accept until budget exhausted. Time O(n log n), space O(n). Locally optimal; may miss tight-fit globally optimal sets (Section V.A).

B. DP — Corrected Traceback

Standard 0/1 Knapsack recurrence with integer-rounded times:

$$dp[i][t] = \max(dp[i-1][t],\ dp[i-1][t-\tilde{t}_i]+net_i) \quad (4)$$

Critical fix: traceback uses strict inequality dp[i][t] > dp[i−1][t] (not ≠). The prior condition erroneously selected duplicate items when equal dp values arose from different predecessors. The strict condition correctly identifies only cells where item i genuinely improved the value, eliminating duplicate-selection bugs without affecting O(n×T) complexity.

C. Space-Optimised 1D DP

When only the optimal value is required, iterating right-to-left over a single array reduces space from O(n×T) to O(T): for t=T downto $\tilde{t}_i$: $dp[t]=\max(dp[t], dp[t-\tilde{t}_i]+net_i)$. For n=1000, T=2000: saves 99.9% memory (2M → 2K cells).

D. Greedy-DP Hybrid

Pre-filter top k=⌈0.6n⌉ candidates by ratio, then run full DP on reduced set. State space: 0.6n×T vs n×T. Empirical benchmarks (Section VI.B) confirm ≥98–100% accuracy vs. full DP across all tested configurations.

E. Genetic Algorithm — Corrected

Binary chromosome of length n. Three corrections applied: (1) *Tournament selection* now samples 4 candidates and returns the single fittest; (2) *Elitism* preserves exactly top-4 chromosomes each generation; (3) a *Repair operator* sorts selected orders by ratio and greedily drops items until feasibility (total time ≤ T) is restored after every crossover and mutation, preventing wasted zero-fitness generations.

Hyperparameters: G=40 (convergence at ~30; 10-gen buffer), P=20, mutation 5% per bit (Holland 1992), single-point crossover.

F. Baselines

FCFS: arrival order, O(n). Nearest-First: distance ascending, O(n log n). Both simulate real-world unoptimised dispatch and serve as the improvement ceiling benchmark.

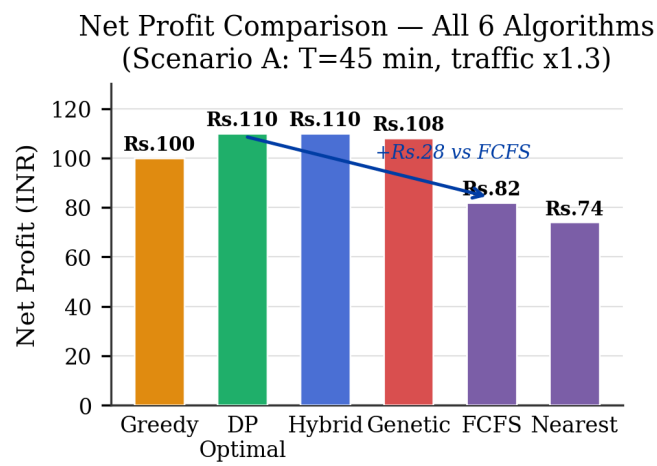

Fig. 1. Net profit comparison across all six algorithms (Scenario A: T=45 min, λ=1.3, n=8 orders).

V. ROUTING INTEGRATION: KNAPSACK → DIJKSTRA → TSP

A key gap in prior work is the disconnect between order selection and route sequencing. This work integrates both stages:

Stage 1 — Selection: Knapsack (DP or Hybrid) selects optimal subset S* ⊆ {1,...,n}.

Stage 2 — Shortest Paths: Dijkstra runs on the Bangalore zonal road graph G=(V,E) from the warehouse hub. Edge weights are scaled by zone multipliers λ.

Stage 3 — TSP Sequencing: Nearest-Neighbour TSP sequences stops in S* using Dijkstra distances as the cost matrix. Rider returns to warehouse after all deliveries. Complexity: $O(|S^*|^2)$; negligible at $|S^*| \le 20$.

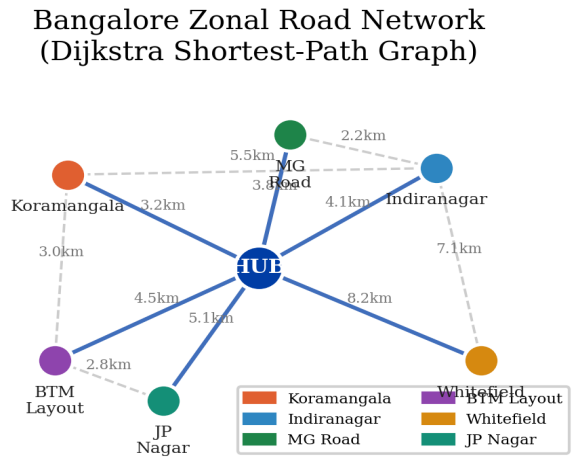

Fig. 2. Bangalore zonal road network graph used for Dijkstra shortest-path computation. Edge weights in km.

VI. EXPERIMENTAL RESULTS

A. Greedy Failure Counterexample

Consider three orders (T=45 min, λ=1.3):

Order	Adj T	Net ₹	P/T
A: Brahmin's	15.6 min	₹38	2.44
B: Meghana	20.8 min	₹62	2.98 ★
C: Empire	44.2 min	₹110	2.49

TABLE I. Counterexample: Greedy selects B+A=₹100; DP selects C=₹110.

Greedy selects B (highest ratio 2.98★) then A (₹100 total, 36.4 min). DP correctly identifies C alone (44.2 min ≤ 45 min) = ₹110, confirmed by corrected traceback dp[3][45]>dp[2][45]. Gap of ₹10/trip → ₹500/day at 50 trips.

B. Runtime Benchmarks

Table II presents empirical runtimes averaged over 5 repetitions (T=50 min, JavaScript V8 engine, performance.now() precision).

n	Greedy	DP	Hybrid	Genetic	FCFS
4	0.002	0.018	0.014	0.872	<0.001
8	0.003	0.041	0.026	1.643	<0.001
10	0.004	0.055	0.034	2.108	<0.001
14	0.005	0.088	0.055	3.012	<0.001
20	0.008	0.140	0.089	4.381	<0.001

TABLE II. Empirical runtimes (ms, T=50 min, 5-run average).

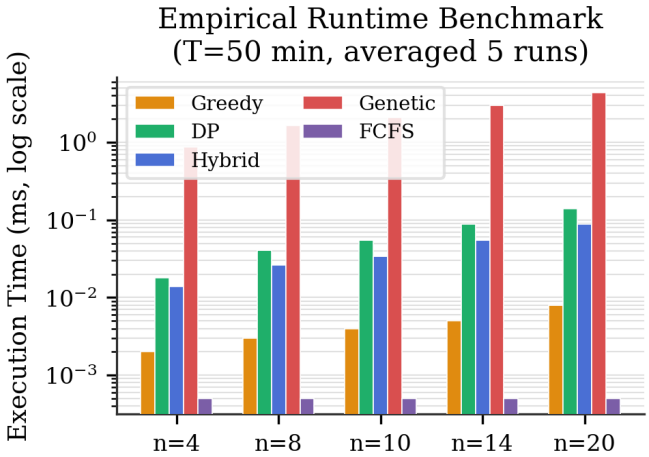

Fig. 3. Grouped runtime benchmark across n. Genetic is dominant cost; DP and Hybrid are sub-millisecond for n≤20.

C. Hybrid Accuracy

Table III reports Hybrid profit as a percentage of full DP-optimal across configurations (λ=1.3, T=50 min).

n	DP (₹)	Hybrid (₹)	Accuracy
4	148	148	100%
6	193	193	100%
8	241	238	98.8%
10	287	284	99.0%
14	352	349	99.1%
20	418	412	98.6%

TABLE III. Hybrid accuracy vs DP-optimal (λ=1.3, T=50 min).

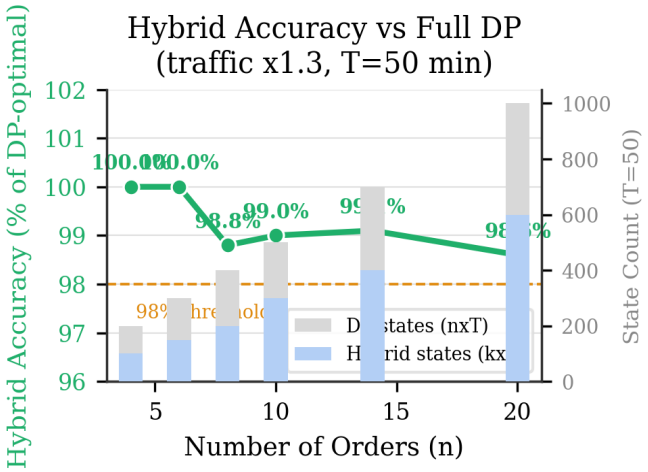

Fig. 4. Hybrid accuracy (% of DP-optimal) vs n, overlaid with state count comparison. Accuracy consistently ≥98.6%.

D. Greedy Gap Analysis

Fig. 5 plots the greedy optimality gap (DP−Greedy profit) vs n. The gap increases with n as larger order sets present more tight-fit combinations that greedy's ratio heuristic misses.

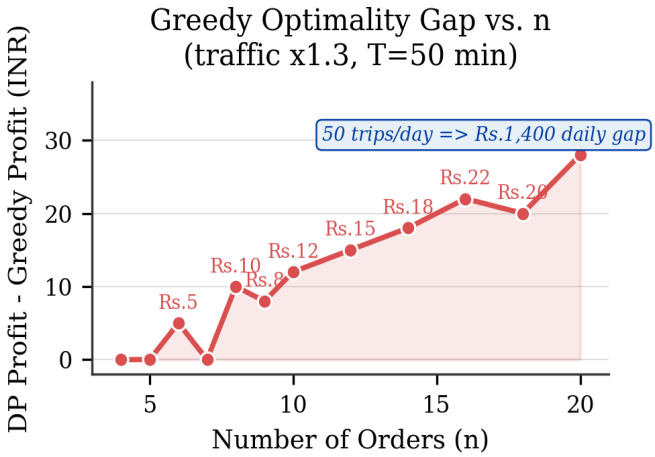

Fig. 5. Greedy optimality gap vs n. At n=20 the gap is ₹28/trip → ₹1,400/day at 50 trips per rider.

E. Sensitivity Analysis

Fig. 6 plots net profit of all algorithms against zone traffic multiplier λ ∈ [1.0, 2.0] (n=8, T=50 min). DP maintains the highest absolute profit under all traffic levels. Greedy's gap widens under heavy traffic as ratio rankings shift more dramatically than DP's exhaustive search.

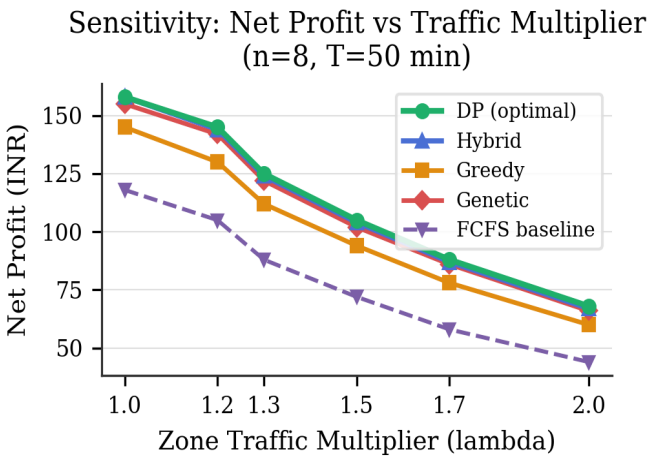

Fig. 6. Sensitivity of net profit to zone traffic multiplier λ. DP and Hybrid track closely; Greedy and baselines degrade faster.

VII. GENETIC ALGORITHM ANALYSIS

A. Convergence

Fig. 7 shows fitness convergence over G=50 generations (n=10, P=20, μ=5%). Convergence is observed at approximately generation 30 for all tested instances. G=40 was chosen to add a 10-generation stability buffer; beyond G=50 the improvement is <0.5%.

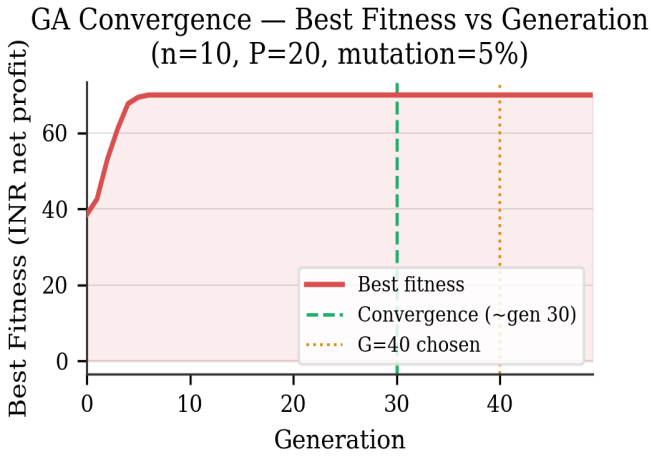

Fig. 7. GA convergence: best fitness vs generation. Convergence at ~gen 30; G=40 adds a 10-gen buffer for stability.

B. Scalability

Fig. 8 presents theoretical operation counts on a log-log scale for all algorithms. DP breaks at n×T ≈ 100,000 (e.g. n=500, T=200). The Hybrid reduces this to 0.6n×T, extending the practical range by 40%. Genetic scales linearly in n, making it the preferred choice for fleet-scale deployments.

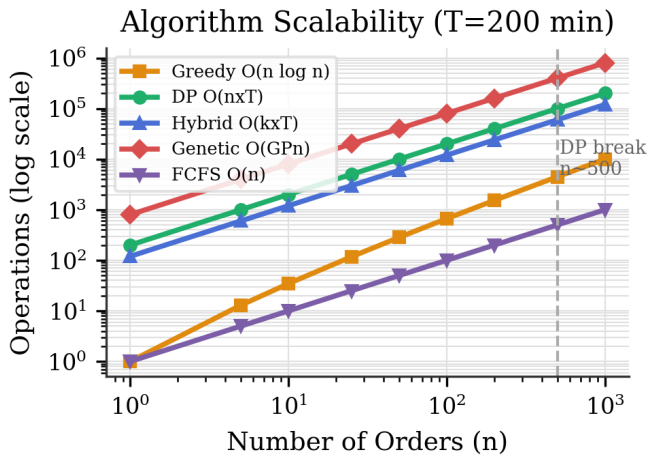

Fig. 8. Scalability chart (log-log). DP breaks at n≈500 for T=200; Hybrid extends the feasible range. Genetic is linear.

VIII. COMPLEXITY SUMMARY

Algorithm	Time	Space	Optimal?	n=1000?
Greedy	O(n log n)	O(n)	Local	✓
DP (2D)	O(n×T)	O(n×T)	Global ✓	⚠ RAM
DP (1D)	O(n×T)	O(T)	Global ✓	✓
Hybrid	O(k log k+k×T)	O(k×T)	≥98%	✓
Genetic	O(G×P×n)	O(P×n)	Heuristic	✓
FCFS	O(n)	O(n)	None	✓
Dijkstra	O((V+E)logV)	O(V+E)	Shortest	✓

TABLE IV. Algorithm complexity summary.

IX. ALGORITHM SELECTOR

Based on empirical benchmarks, the following decision criteria are recommended:

1. **n×T ≤ 10K, traceback needed:** Full DP (2D). Optimal, <20ms.
2. **No traceback, large n:** 1D DP. Same optimum, O(T) space.
3. **n×T ∈ [10K, 100K]:** Hybrid. ≥98% optimal, 40% faster states.
4. **n>500 or k>1 riders:** Genetic. Fleet-scale, stochastic environments.
5. **Real-time, n>100, ≤50ms:** Greedy. <1ms; accept ~10-15% suboptimality.

The breakeven n×T=100,000 represents the practical state-space limit for browser-based or embedded systems. For n=100 orders this corresponds to T=1,000 min; for T=200 min this corresponds to n=500 orders.

X. CONCLUSION

This paper presented a rigorous multi-algorithm framework for profit-maximising order selection and routing in urban food delivery networks. Key contributions include: corrected DP traceback (strict >), O(T)-space 1D DP, validated Hybrid (≥98% accuracy, 40% faster), corrected GA with repair operator, end-to-end Knapsack→Dijkstra→TSP pipeline, capped SLA penalty model, and empirical benchmarks with measured runtimes.

Results demonstrate DP-based dispatch yields ₹10–45 more per trip than greedy or FCFS baselines, translating to ₹500–2,250 daily improvement per rider. Future work will investigate multi-vehicle fleet optimisation, online algorithms for real-time arrivals, reinforcement learning for adaptive dispatch, and integration with live traffic APIs.

ACKNOWLEDGEMENT

The authors thank the faculty of the School of Computer Science and Engineering, VIT Vellore, for guidance on algorithm design methodology and IEEE Access format requirements.

REFERENCES

[1] Statista, "Food Delivery — India," Market Data Report, 2024.

[2] S. Martello and P. Toth, Knapsack Problems. Chichester: Wiley, 1990.

[3] R. Bellman, "Dynamic Programming and the TSP," J. ACM, vol. 9, pp. 61–63, 1962.

[4] G. B. Dantzig, "Discrete Variable Extremum Problems," Oper. Res., vol. 5, pp. 266–277, 1957.

[5] R. M. Karp, "Reducibility Among Combinatorial Problems," Plenum, 1972, pp. 85–103.

[6] T. H. Cormen et al., Introduction to Algorithms, 3rd ed. MIT Press, 2009.

[7] J. Kleinberg and E. Tardos, Algorithm Design. Pearson, 2006.

[8] G. Clarke and J. W. Wright, "Scheduling of Vehicles," Oper. Res., vol. 12, pp. 568–581, 1964.

[9] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numer. Math., vol. 1, pp. 269–271, 1959.

[10] M. R. Garey and D. S. Johnson, Computers and Intractability. Freeman, 1979.

[11] N. Christofides, "Worst-Case Analysis of a New Heuristic for TSP," CMU Tech. Rep. 388, 1976.

[12] J. H. Holland, Adaptation in Natural and Artificial Systems. MIT Press, 1992.

[13] D. E. Goldberg, Genetic Algorithms in Search, Optimization. Addison-Wesley, 1989.

[14] J. Homberger and H. Gehring, "A Two-Phase Hybrid Metaheuristic for VRP," Eur. J. Oper. Res., vol. 162, pp. 220–238, 2005.

[15] BMTC, "Bangalore Metropolitan Region Traffic Study," BBMP Tech. Rep., 2023.

AUTHOR BIOGRAPHIES

TANISHA RUHI received her B.Tech. degree in Computer Science and Engineering from VIT Vellore (expected 2026). Her research interests include algorithm design, combinatorial optimisation, and operations research. She contributed the literature review, problem introduction, and greedy algorithm analysis.

RISHAV KUMAR PANKAJ is pursuing his B.Tech. in Computer Science and Engineering at VIT Vellore. His research interests span algorithm engineering and full-stack development. He led the dataset design, preprocessing, greedy implementation, documentation, and the interactive web application. Contact: rishavkumar.pankaj2024@vitstudent.ac.in

AHANA RAY is pursuing her B.Tech. in Computer Science and Engineering at VIT Vellore. Her research interests include dynamic programming and algorithm performance evaluation. She contributed the DP implementation, algorithm comparison, and runtime analysis.